

DAY 3 – DOCUMENTATION

Name: Moturi Lalitha Sowjanya

Team: 4

Date: 03-06-26 / 04-06-26

1. OBJECTIVE

To study the complete Retrieval-Augmented Generation (RAG) pipeline and understand how different components work together to retrieve relevant information and generate accurate responses.

2. RESEARCH TOPICS

2.1. What Problems RAG Solves

1. Outdated Knowledge

- LLMs may not know the latest information.
- They can only answer based on the data used during training.

2. Hallucinations

- LLMs may generate incorrect or made-up answers.
- Sometimes they answer confidently even when the information is not available.

3. No Access to Private Data

- LLMs cannot access private documents such as PDFs, company records, notes, or college documents unless provided.

4. Expensive Fine-Tuning

- Updating an LLM by retraining it with new data is costly and time-consuming.

5. No Citations

- LLMs often cannot show the exact source of the information used to generate an answer.
-

2.2. RAG Components

1. Data Sources

Data sources are the knowledge sources used by a RAG system.

Examples:

- PDFs • Documents • Web Pages • Databases
- Notion • Google Drive • Local Text Files • College Policy Documents

2. Document Loader

A document loader reads data from different sources and loads it into the RAG system.

Types:

- PDF Loader • CSV Loader • Text Loader
- Web Loader • Database Loader

3. Text Cleaning

Text cleaning removes unnecessary information from extracted text.

Example:

Before:

Page 1 | Footer | Website Link

After:

Clean paragraph content

4. Chunking

Chunking divides large documents into smaller sections called chunks.

Example:

Large Document

↓

Chunk 1

↓

Chunk 2

↓

Chunk N

5. Embedding Model

An embedding model converts text into numerical vectors.

Example:

"AI is transforming education"

↓

[0.12, -0.45, 0.89, ...]

6. Vector Database

A vector database stores embeddings and performs similarity searches.

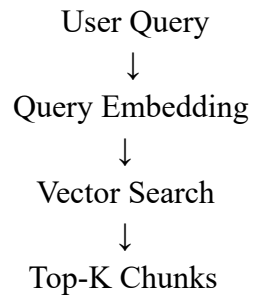
Examples:

- FAISS • ChromaDB
- Pinecone • Weaviate

7. Retriever

The retriever finds the most relevant chunks for a user query.

Workflow:



8. Similarity Search

Similarity search compares the query vector with stored document vectors.

Example:

Query:

“How many books can a student borrow?”

Retrieved Chunks:

1. Students can borrow up to 4 books at a time.
2. Library membership rules.
3. Book return and fine policy.

9. Prompt Augmentation

Retrieved context is added to the prompt before sending it to the LLM.

Structure:

Instruction + Retrieved Context + User Question

10. LLM Generation

The LLM generates the answer using the provided context.

Input:

- Instruction
- Retrieved Context
- User Question

Output:

- Generated Answer

11. Final Response

The final answer returned to the user.

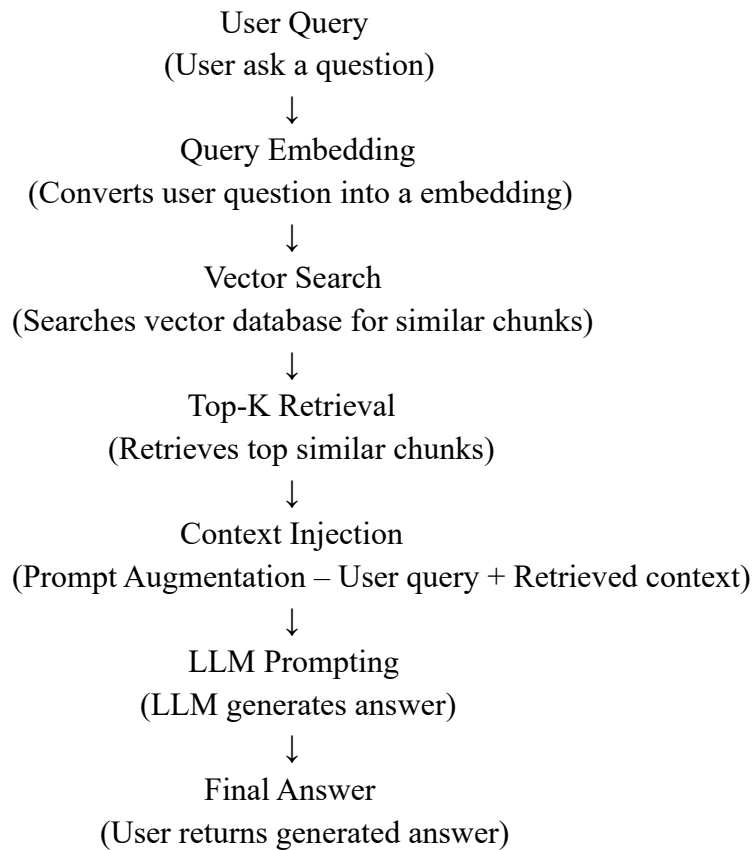
A Good Response Should Include:

- Direct Answer
 - Supporting Details
 - Source/Citation
 - Uncertainty if information is missing
-

2.3. RAG Data Flow

The Complete Data Flow describes how a RAG system processes a user query and generates an answer.

Workflow:



2.4. Retrieval Methods/Techniques

Retrieval Methods/Techniques determines how a RAG system finds the most relevant document chunks for a user query.

1. Cosine Similarity

Cosine Similarity measures the similarity between two vectors by calculating the angle between them.

Results:

- High Similarity = Similar Meaning
- Low Similarity = Different Meaning

Example:

Query:
"What is RAG?"

Similar Chunk:

"Retrieval-Augmented Generation combines retrieval with LLM generation.

2. Dense Retrieval

- Dense Retrieval uses embeddings to search based on semantic meaning.
- Finds relevant information even when exact words are not present.

Example:

Query:

"College food system"

Retrieved Chunk:

"Canteen and mess management"

3. Sparse Retrieval

- Sparse Retrieval uses keyword matching to find relevant documents.
- Find documents containing exact keywords

Example:

Query:

"Form 16A"

Retrieved Chunk:

Document containing "Form 16A"

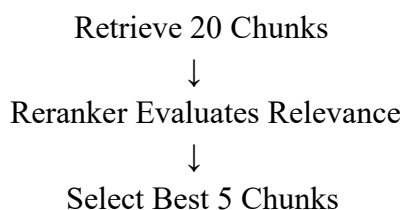
4. Hybrid Search

- Hybrid Search combines Dense Retrieval and Sparse Retrieval.
- Improve retrieval accuracy.

5. Reranking

- Reranking is the process of reordering retrieved chunks according to relevance.
- Improve context quality before sending it to the LLM.

Workflow:



2.5. Chunking Methods/Techniques

Chunking is the process of dividing large documents into smaller, meaningful pieces called chunks.

1. Fixed Chunking

Splits text into chunks of a fixed size.

Example:

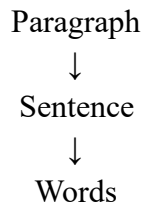
Every 500 tokens = 1 chunk

Limitation:

- May split sentences or ideas in the middle.
- Can lose context.

2. Recursive Chunking

Splits text intelligently using document structure.

Process:**3. Semantic Chunking**

Splits documents based on meaning or topics.

Example:

Topic A: Hostel Rules

Topic B: Attendance Policy

Topic C: Exam Regulations

Each topic becomes a separate chunk.

4. Sliding Window Chunking

Creates overlapping chunks to preserve context between neighboring chunks.

Example:

Chunk 1: Lines 1–10

Chunk 2: Lines 8–18

Chunk 3: Lines 16–26

2.6. Embeddings

- Embeddings are numerical representations of the meaning of text.
- Similar words have similar vectors.
- Different words have different vectors.

Example:

"Education" → Vector

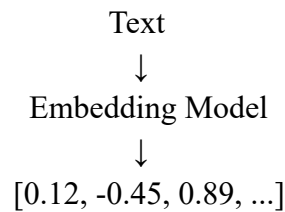
"Exams" → Nearby Vector

"Parents" → Far Vector

2.6.1. Vector Dimensions

A dimension is one value in a vector.

Example:

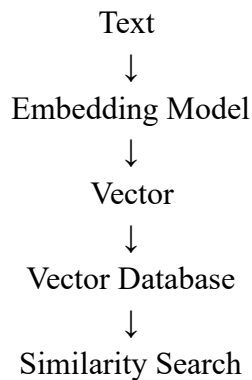


Each number represents one dimension.

2.7. Vector Databases

A Vector Database is a specialized database used to store embeddings (vectors) and perform similarity searches efficiently.

Workflow:



Examples:

1. FAISS

- Facebook AI Similarity Search
- A fast and lightweight vector search library developed by Meta.

2. ChromaDB

A beginner-friendly vector database designed for AI and RAG applications.

3. Pinecone

A managed cloud vector database.

4. Weaviate

A vector database with built-in semantic search capabilities.

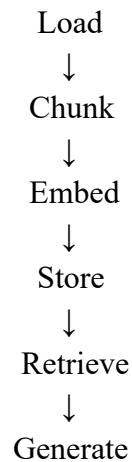
2.8. Advanced RAG Architectures

Advanced RAG Architectures are improved versions of basic RAG systems designed to provide better retrieval accuracy, reasoning, and scalability.

1. Naive RAG

The simplest form of a RAG system.

Workflow:



2. Advanced RAG

An improved version of Naive RAG with additional retrieval techniques.

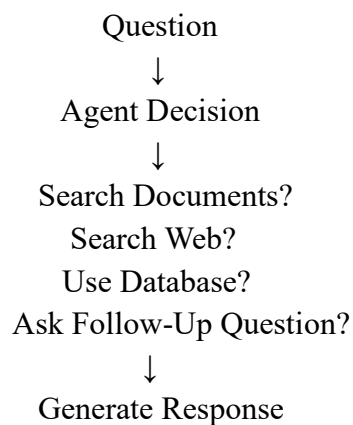
Features:

- Query Rewriting
- Hybrid Search
- Reranking
- Metadata Filtering
- Evaluation

3. Agentic RAG

A RAG system controlled by an AI agent.

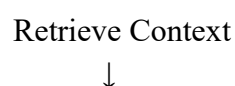
Workflow:

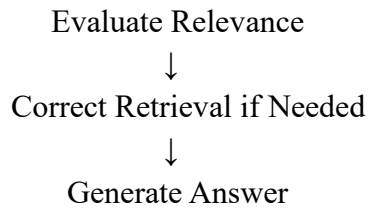


4. Corrective RAG

A RAG system that checks whether retrieved information is useful before generating an answer.

Workflow:





5. Graph RAG

A RAG system that uses knowledge graphs to represent relationships between entities.

Structure:

Entities + Relationships + Documents

6. Multi-Modal RAG

A RAG system that retrieves information from multiple data types.

Supported Data:

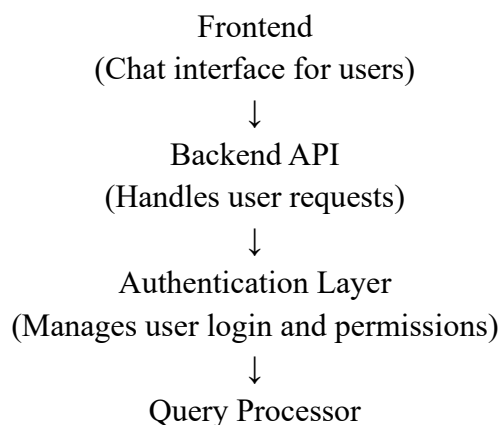
- Text
- Images
- Audio
- Video
- Tables
- PDFs

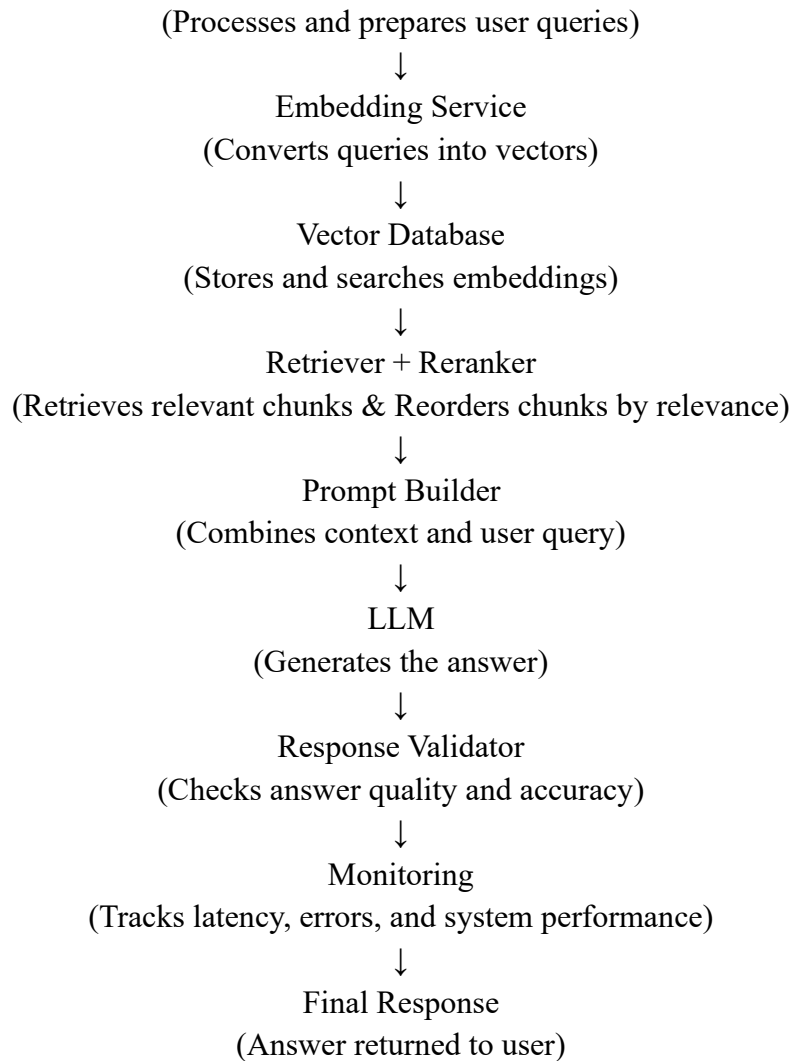
2.9. Important Metrics

1. **Retrieval Precision** – Are retrieved chunks relevant?
2. **Retrieval Recall** – Did the system retrieve all useful chunks?
3. **Faithfulness** – Is the answer based on the retrieved context?
4. **Latency** – How quickly is the answer generated?
5. **Cost Per Query** – Cost of processing one query.
6. **Hallucination Rate** – Frequency of incorrect or invented answers.

2.10. Real-World Production Architecture

Workflow:





3. Tasks Completed / What I Learned

- Researched the complete RAG pipeline from document ingestion to answer generation.
- Learned how retrieval improves the accuracy of LLM responses.
- Studied chunking techniques and their impact on retrieval quality.
- Understood embeddings and vector databases used in RAG systems.
- Explored retrieval methods such as dense retrieval, sparse retrieval, hybrid search, and reranking.
- Learned about advanced RAG architectures including Agentic RAG, Graph RAG, and Multi-Modal RAG.
- Understood common RAG challenges and their solutions.

4. Conclusion

The complete RAG pipeline was studied in detail, covering document processing, retrieval mechanisms, embeddings, vector databases, advanced architectures, and real-

world deployment considerations. This research provided a clear understanding of how RAG systems retrieve relevant information and generate context-aware responses.